The potential for spurious wakeups means that it’s unspecified how many times `pred` will be called. `pred` will always be invoked with the mutex referenced by `lock` locked, and the function shall return if (and only if) an evaluation of `(bool)pred()` returns `true`.

Synchronization

Calls to `notify_one(), notify_all(), wait(), wait_for()` and `wait_until()` on a single `std::condition_variable` instance are serialized. A call to `notify_one()` or `notify_all()` will only wake threads that started waiting *prior* to that call.

Std::Condition_Variable::Wait_For Member Function

Waits until the `std::condition_variable` is notified by a call to `notify_one()` or `notify_all()`, or until a specified time period has elapsed or the thread is woken spuriously.

Declaration

```
template<typename Rep,typename Period>
cv_status wait_for(
    std::unique_lock<std::mutex>& lock,
    std::chrono::duration<Rep,Period> const& relative_time);
```

Preconditions

`lock.owns_lock()` is `true`, and the lock is owned by the calling thread.

Effects

Atomically unlocks the supplied `lock` object and block until the thread is woken by a call to `notify_one()` or `notify_all()` by another